

End-to-End Performance Engineering of Google Online Boutique

Muhammad Sohail (7178621)

Syed Umair Anwar (7172248)

July 17, 2026

Course: SOFTWARE PERFORMANCE ENGINEERING 2025–2026

Case Study: End-to-End Case Study (Project 7)

Benchmark: Google Online Boutique (microservices-demo)

Fork: <https://github.com/msohailse/microservices-demo-spe>

Contents

1. Project Overview	4
2. System Architecture	4
3. Setup and Deployment	4
3.1. Environment	4
3.2. Deployment	4
3.3. Fork Fix: Resource Limits for Minikube Stability	5
3.4. Monitoring Stack	5
3.5. Distributed Tracing (Jaeger + Istio)	6
4. Workload Measurement	8
4.1. Load Generation	8
4.2. Metrics Collected	8
4.3. Consolidated Measurement Script (<code>scripts/measure.sh</code>)	10
5. Analytical Modelling (QN/LQN)	11
5.1. Model Construction	11
5.2. Open QN Product-Form Solution	13
5.3. Validation	13
5.4. Formal Study: JMT JSIM Simulation	14
6. What-If Analysis	15
6.1. Model and Saturation Point	15
6.2. Settled Baseline Measurement ($N = 10$)	15
6.3. Stress Tests	16
6.4. Discussion of Results	16
6.5. Cost Analysis	17
7. Optimization	18
7.1. Bottlenecks Identified	18
7.2. Improvements Applied	18
7.3. Measured Performance Results (M/M/2 Frontend)	19

1. Project Overview

This report documents a comprehensive performance engineering study of the Google Online Boutique, a cloud-native microservice benchmark application. The study covers the full pipeline: deployment, workload measurement, analytical modelling via Queueing Networks (QN/LQN), and performance optimization.

2. System Architecture

Google Online Boutique consists of 11 microservices written in multiple languages, deployed on Kubernetes. The services communicate via gRPC.

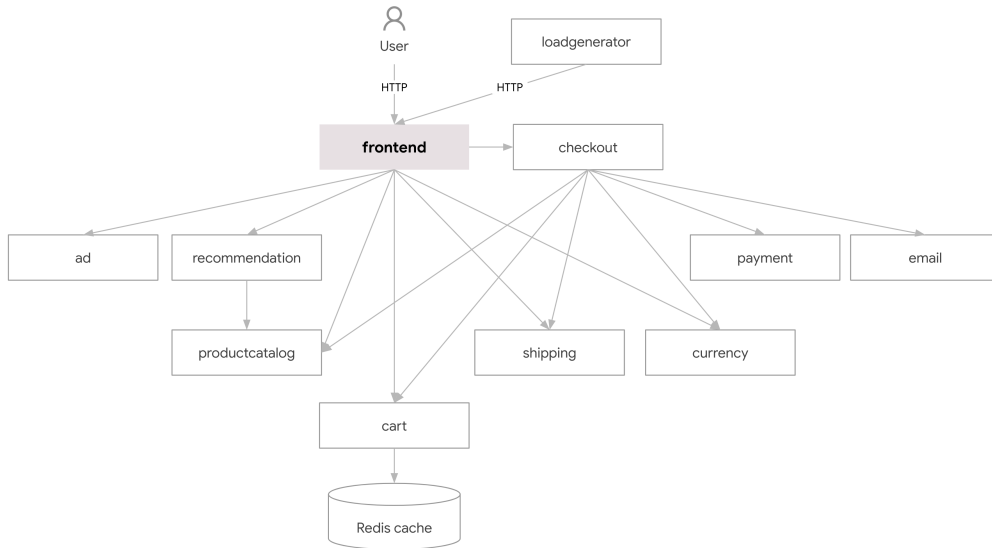


Figure 1: Google Online Boutique architecture (source: GoogleCloudPlatform/microservices-demo)

3. Setup and Deployment

3.1. Environment

- Minikube with Docker driver, 6 CPUs, 9 GB memory
- Helm for monitoring stack deployment

3.2. Deployment

```
minikube start --driver=docker --cpus=6 --memory=9g
kubectl apply -f release/kubernetes-manifests.yaml
```

```
Done. To access things:
App: minikube service frontend-external
Grafana: kubectl port-forward -n monitoring svc/monitoring-grafana 3000:80
Prometheus: kubectl port-forward -n monitoring svc/monitoring-kube-prometheus-prometheus 9090:9090
Jaeger: kubectl port-forward -n monitoring svc/jaeger-query 16686:16686
Grafana login: admin / prom-operator
```

Figure 2: `./scripts/deploy.sh` completion output: all services deployed and port-forward commands printed for Grafana, Prometheus, and Jaeger.

3.3. Fork Fix: Resource Limits for Minikube Stability

The study uses a fork of the benchmark that also includes all measurement and deployment scripts (<https://github.com/msohailse/microservices-demo-spe>). The upstream manifests set CPU limits (300m for `cartservice`, 150m for `currencyservice`) tuned for GKE node pools, which caused `CrashLoopBackOff` on Minikube: both services are CPU-intensive at startup (`cartservice` runs .NET JIT warmup; `currencyservice` has a single-threaded Node.js event loop), and the original caps throttled them before they could respond to their gRPC liveness probes. The fork increases these limits to 600m and 300m respectively and raises memory to 256Mi, giving each container the burst headroom to pass health checks reliably. All performance measurements in this report are taken after this fix is in place. While this infrastructure fix restores system stability, it increases the baseline computational cost of the cluster by allocating an additional 300m CPU to `cartservice` and 150m CPU to `currencyservice`.

Metric	Before fix (upstream limits)	After fix (increased limits)
System status	<code>CrashLoopBackOff</code>	Stable
System mean R (ms)	Failing	38.6
System p95 (ms)	Timeout / 500 Error	97.8

Table 1: Effect of increasing CPU limits on `cartservice` (300 m $\rightarrow$ 600 m) and `currencyservice` (150 m $\rightarrow$ 300 m). Before values reflect the upstream configuration where liveness probe failures caused `CrashLoopBackOff`; after values are empirical measurements from Prometheus at baseline load ($N=10$, RATE=1).

3.4. Monitoring Stack

Prometheus and Grafana deployed inside the cluster via Helm (kube-prometheus-stack). Prometheus auto-discovers all pods via Kubernetes service discovery. Grafana connects to Prometheus for dashboards. Port-forwarding tunnels browser access into the minikube network.

```
./scripts/monitoring-setup.sh
```

Access Grafana via port-forward at `localhost:3000`, username `admin`. The chart generates a random admin password on each fresh install rather than a fixed default, retrievable via:

```
kubectl --namespace monitoring get secrets monitoring-  
grafana \  
-o jsonpath="{.data.admin-password}" | base64 -d; echo
```

3.5. Distributed Tracing (Jaeger + Istio)

Per the assignment’s Guidelines and Best Practices, tracing is required both for dependency-graph extraction and to capture routing probabilities and per-service demands used in the QN/LQN model. Rather than instrumenting each of the 11 services individually with OpenTelemetry SDK calls, tracing is obtained for free from the service mesh already used in this deployment: Istio injects an Envoy sidecar proxy into every pod, and each sidecar reports a trace span for every request it forwards, with no application code changes.

Deployment order matters. Istio’s sidecar injection is decided once, at pod-creation time, based on a namespace label. The pipeline therefore enables Istio and applies the tracing configuration *before* the application pods exist, so every one of the 11 services comes up already wrapped in a sidecar and already pointed at a tracing backend:

```
./scripts/monitoring-setup.sh # (1)  
Istio + injection label + Prometheus/Grafana  
kubectl apply -f monitoring-manifests/monitoring.yaml #  
(2) Jaeger + Istio Telemetry resource  
kubectl apply -f release/kubernetes-manifests.yaml #  
(3) 11 microservices (sidecars auto-injected)  
kubectl apply -f release/istio-manifests.yaml #  
(4) Gateway/HTTPRoute exposing the frontend
```

Components:

- **Jaeger** (`jaegertracing/all-in-one:1.60`), distributed tracing system deployed in the `monitoring` namespace.
- **jaeger-query**, a Kubernetes `Service` exposing port 16686, giving Grafana and the browser a stable DNS name for Jaeger’s UI/API instead of a pod IP.
- **Envoy sidecar (the “Istio Proxy”)**, injected into every application pod. It does the physical work of capturing each request/response and forwarding the resulting span across the network to Jaeger; the application containers are unaware this is happening.

- **Zipkin format**, the common wire format both sides speak: Envoy has a built-in Zipkin trace exporter, and Jaeger's all-in-one image exposes a Zipkin-compatible receiver on port 9411. An Istio **Telemetry** resource registers this Zipkin endpoint as the mesh's tracing provider (at 100% sampling, since a benchmarking study needs every trace, not a 1% production sample).

Grafana/Prometheus wiring. `monitoring-manifests/override_values.yaml` is passed to the `kube-prometheus-stack` Helm release by `scripts/monitoring-setup.sh`. It adds Jaeger as a Grafana datasource and sets a 6-hour Prometheus retention window.

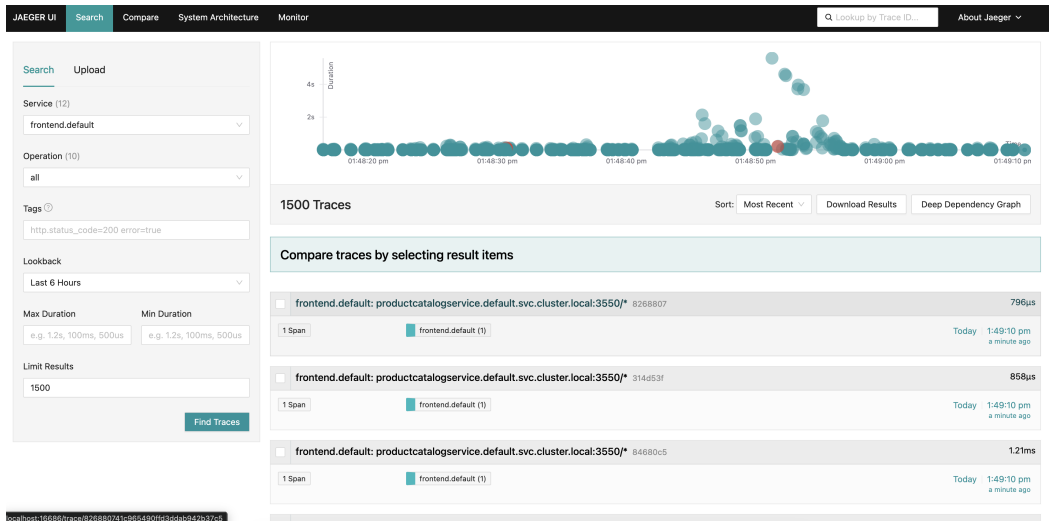


Figure 3: Jaeger traces for `frontend`: 100% sampling active under `loadgenerator` traffic.

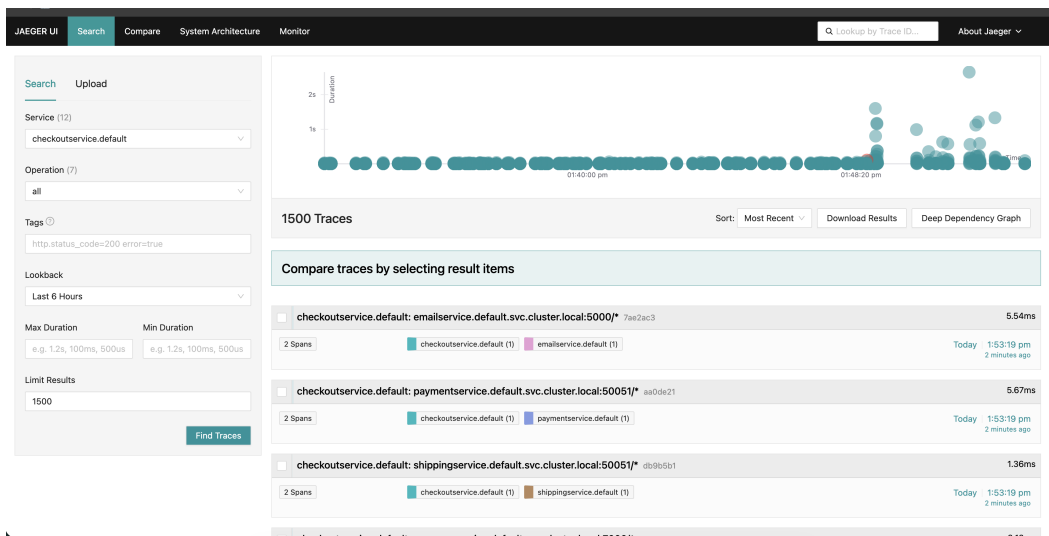


Figure 4: Jaeger traces for `checkoutservice`: lower volume consistent with $V_{\text{checkoutservice}} = 0.040$.

4. Workload Measurement

4.1. Load Generation

Workload is generated by `loadgenerator`, a Locust-based deployment included in the upstream manifests. Four load levels were applied across this study:

- Baseline: USERS=10, RATE=1
- Stress Test 1: USERS=50, RATE=10
- Stress Test 2: USERS=100, RATE=10
- Stress Test 3: USERS=200, RATE=20

4.2. Metrics Collected

All metrics originate from Envoy sidecars scraped via Prometheus — no application-level instrumentation was needed.

Throughput / arrival rate (λ):

```
sum by (destination_workload) (rate(istio_requests_total[2m]))
```

Service	λ_i (req/s)	S_{meas} (ms)	S_{net} (ms)	V_i	p95 (ms)
productcatalogservice	29.10	0.236	0.236	5.424	4.1
currencyservice	17.93	1.855	1.855	3.019	5.0
cartservice	5.87	1.876	1.876	0.968	4.9
frontend	5.77	32.837	17.126	1.000	97.8
recommendationservice	4.00	8.985	8.749	0.716	24.3
adservice	3.37	1.305	1.305	0.567	4.9
shippingservice	1.97	0.659	0.659	0.292	4.6
paymentservice	0.33	3.462	3.462	0.037	17.5
checkoutservice	0.33	31.438	12.641	0.040	77.5
emailservice	0.33	3.462	3.462	0.036	17.5

Table 2: Baseline per-service measurements (USERS=10, $\lambda_{\text{root}} = 5.22$ req/s): per-service arrival rate λ_i , measured residence time S_{meas} , net service time S_{net} (downstream RPC wait subtracted via $S_{\text{net}}[s] = S_{\text{meas}}[s] - \sum_c f_{sc} S_{\text{net}}[c]$), visit ratio $V_i = \lambda_i / \lambda_{\text{root}}$, and 95th-percentile latency from Prometheus `histogram_quantile`.

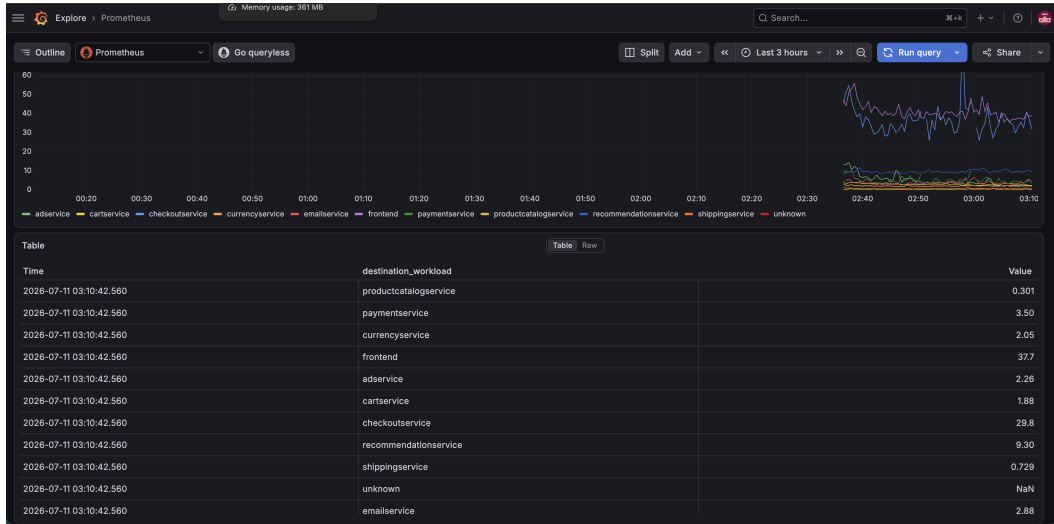


Figure 5: All-services arrival rate (Prometheus API)

Mean service time:

```
sum by (destination_workload) (rate(
  istio_request_duration_milliseconds_sum[2m]))
/ sum by (destination_workload) (rate(
  istio_request_duration_milliseconds_count[2m]))
```

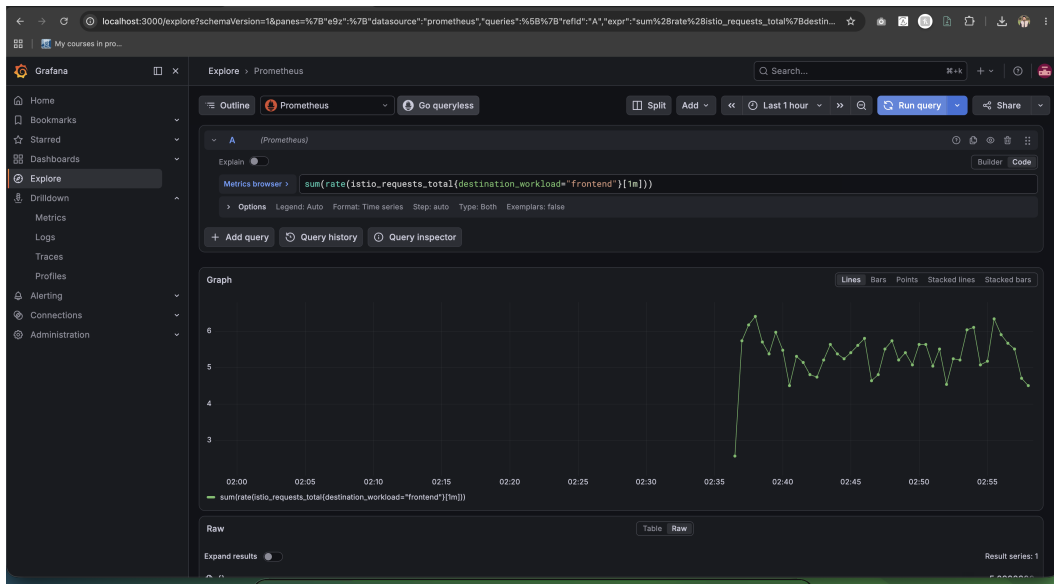


Figure 6: Frontend request throughput over time (Grafana Explore)

Latency (p95):

```
histogram_quantile(0.95, sum by (destination_workload, le)
  (rate(istio_request_duration_milliseconds_bucket[2m])))
```

p95 is derived from Prometheus histogram buckets (not Jaeger traces) and is captured per service by `scripts/measure.sh` at each load level. Per-service baseline values appear in Table 2; stress-test results are in Table 6.

4.3. Consolidated Measurement Script (`scripts/measure.sh`)

`scripts/measure.sh` consolidates all model inputs into one command. It queries Prometheus for service times and throughput, and Jaeger `/api/dependencies` for routing probabilities and visit ratios, producing all inputs needed to calibrate the JMT model.

```
./scripts/measure.sh
```

```
=====
1. ROUTING PROBABILITIES -- Jaeger
=====

From checkoutservice (total calls: 3892):
-> currencyservice      p=0.289
-> productcatalogservice p=0.203
-> shippingservice      p=0.170
-> cartservice          p=0.170
-> paymentservice       p=0.085
-> emailservice         p=0.084

From frontend (total calls: 89749):
-> productcatalogservice p=0.465
-> currencyservice       p=0.291
-> cartservice           p=0.090
-> recommendationservice p=0.071
-> adservice             p=0.057
-> shippingservice       p=0.022
-> checkoutservice       p=0.004

From loadgenerator (total calls: 9005):
-> frontend              p=1.000

From recommendationservice (total calls: 6420):
-> productcatalogservice p=1.000

=====
2. JMT INPUT TABLE (mean service time in seconds)
=====
```

Station	Mean (s)	lambda=1/S
loadgenerator (source)	0.18634	5.36657716215223
frontend	0.03737	26.76
checkoutservice	0.03394	29.47
recommendationservice	0.00942	106.11
emailservice	0.00376	265.65
paymentservice	0.00361	276.82
adservice	0.00260	384.21
cartservice	0.00197	506.40
currencyservice	0.00191	524.32
shippingservice	0.00058	1731.59
productcatalogservice	0.00027	3696.19

Figure 7: `scripts/measure.sh` output: routing probabilities (Jaeger) and net service times (Prometheus).

From	To	p
loadgenerator	frontend	1.000
frontend	productcatalogservice	0.465
frontend	currencyservice	0.291
frontend	cartservice	0.090
frontend	recommendationservice	0.072
frontend	adservice	0.057
frontend	shippingservice	0.022
frontend	checkoutservice	0.004
recommendationservice	productcatalogservice	1.000
checkoutservice	currencyservice	0.289
checkoutservice	productcatalogservice	0.203
checkoutservice	cartservice	0.169
checkoutservice	shippingservice	0.169
checkoutservice	paymentservice	0.085
checkoutservice	emailservice	0.084

Table 3: Per-edge routing probabilities: each parent’s outbound calls normalized to sum to 1, from Jaeger `/api/dependencies` (102,448 frontend calls, 4,458 checkout calls). Distinct from visit ratios V_i (Table 2): these are single-step transition probabilities, not per-root visit counts.

5. Analytical Modelling (QN/LQN)

5.1. Model Construction

The model requires Istio sidecar traces (for call topology) and Prometheus metrics (for service times). `scripts/measure.sh` collects both in one command.

Visit ratios V_i are extracted from Jaeger’s `/api/dependencies` endpoint, which returns aggregate `parent`→`child` call counts across all traces in memory:

```
curl "http://localhost:16686/api/dependencies?endTs=<
now_ms>&lookback=3600000"
```

V_i = total calls into service i / root entry count (`loadgenerator`→`frontend`). Measured values are in Table 2.

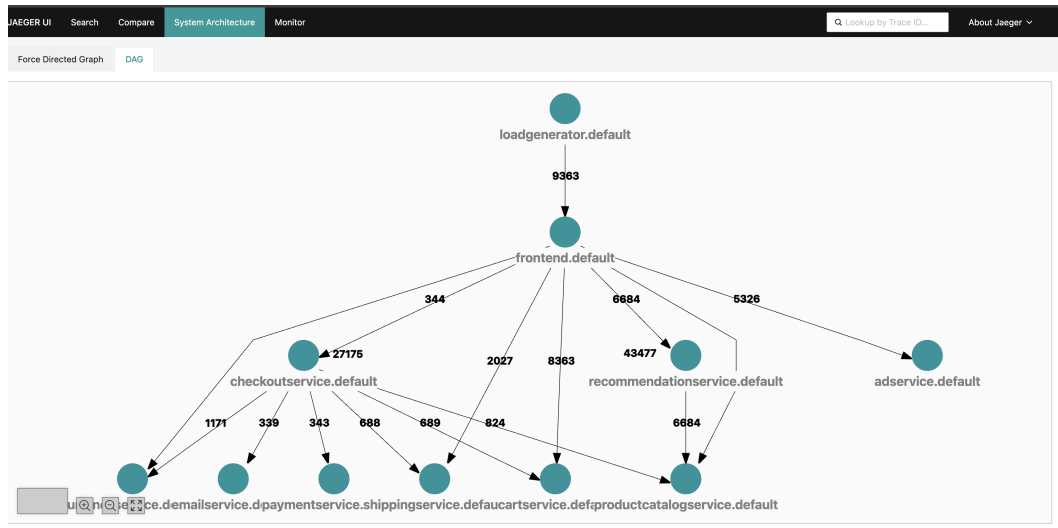


Figure 8: Jaeger dependency graph: service call topology extracted from Istio sidecar traces.

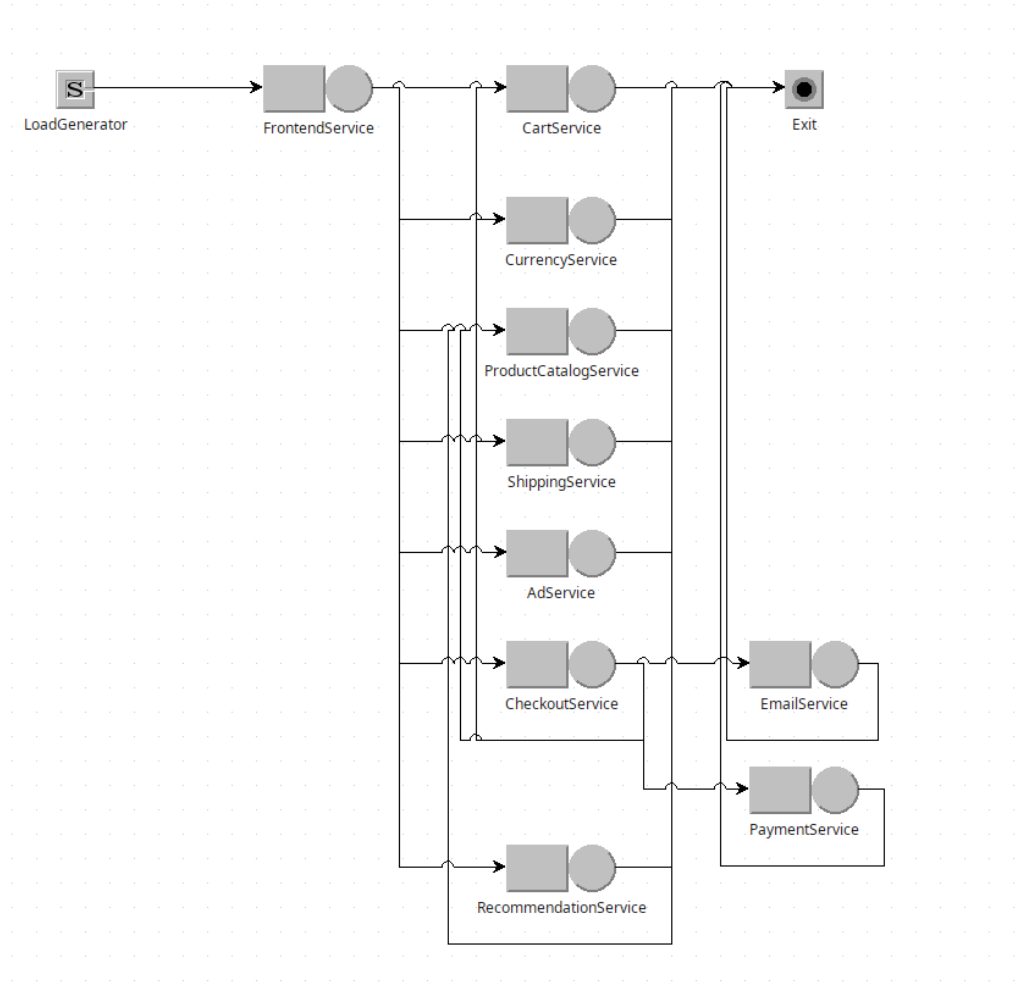


Figure 9: JMT open Jackson network model (jmt-model/SPEProject.jsim): Source $\rightarrow$ frontend $\rightarrow$ weighted fan-out $\rightarrow$ Sink, with routing probabilities from Table 3.

5.2. Open QN Product-Form Solution

For an open Jackson network the mean residence time at each station follows the product-form solution directly. Service demand $D_i = V_i \times S_{\text{net},i}$; utilization $U_i = \lambda \cdot D_i$; station residence time $R_i = D_i / (1 - U_i)$. At baseline ($V_{\text{frontend}} = 1$, $S_{\text{net},\text{frontend}} = 17.13 \text{ ms}$, $\lambda = 5.22 \text{ req/s}$) all utilizations are well under 1 and frontend is the bottleneck at $U = 9.6\%$. Per-node results are shown in Table 5.

5.3. Validation

The model predicts $R = 35.87 \text{ ms}$; measured mean is 38.62 ms , a **7% underestimate**. The correction $S_{\text{net}}[s] = S_{\text{measured}}[s] - \sum_c f_{sc} S_{\text{net}}[c]$ removes downstream RPC wait from upstream service times, which reduces error from $\sim 2\times$ (naive

model) to $<10\%$.

The remaining gap is explained by the parallel-call approximation: the correction assumes downstream calls are sequential, but **frontend** fires some in parallel, so actual wait is less than the sum, leaving $S_{\text{net},\text{frontend}}$ slightly underestimated. The flat open QN (vs. a true LQN) is a deliberate scope choice; the checkout path carries only $V_{\text{checkoutservice}} = 0.040$ of traffic, so the layering error is small system-wide.

5.4. Formal Study: JMT JSIM Simulation

The open Jackson network is modelled formally in JMT JSIM (`jmt-model/SPEProject-Test2.jsim`), one open-class Source, one Sink, and one M/M/1 station per service with service rate $1/S_{\text{net},i}$ and the routing probabilities from Table 3. JMT JSIM is the primary model tool for this study; the closed-form product-form formula serves as an analytical cross-check. At Test 2 conditions ($\lambda = 51.46$ req/s) the two agree closely (Table 4), confirming the model calibration.

Metric	JSIM (formal model)	Formula (cross-check)	Measured
System R (ms)	158.8	180.2	199.3
Throughput (req/s)	51.21	51.46	51.46

Table 4: JMT JSIM (formal study) vs. analytical product-form formula vs. measured at Test 2 ($\lambda = 51.46$ req/s). JSIM underestimates measured R by 20%; the formula by 9.6%. Bottleneck utilization $U_{\text{fe}} = 0.888$ at Test 2 is derived from $\lambda \cdot D_{\text{fe}}$ and reported in Table 6.

Both model variants underestimate measured R , for the same structural reason: both assume exponential (M/M/1) service times. In practice **frontend**’s service-time distribution is heavier-tailed than exponential, so actual mean residence time exceeds the M/M/1 prediction. The formula’s smaller gap (9.6% vs 20%) arises because the S_{net} correction is computed from measured means and indirectly absorbs some distributional variance before the formula is applied; JSIM takes $S_{\text{net},i}$ as the exponential mean directly and does not benefit from this. Both models are useful: JSIM is the formal tool producing per-station queue lengths (N_i via Little’s Law) and confidence intervals; the formula gives rapid closed-form cross-checks and supports the what-if $R(\lambda)$ curve.

6. What-If Analysis

6.1. Model and Saturation Point

The JMT JSIM model (`jmt-model/SPEProject-Test2.jsimg`) is used as the formal prediction tool for this section. The analytical product-form formula serves as a closed-form cross-check; both agree closely at Test 2 conditions (Table 4), so the formula traces the continuous $R(\lambda)$ curve shown below:

$$R(\lambda) = \sum_i \frac{D_i}{1 - \lambda D_i}, \quad \lambda < \lambda_{\max} = \frac{1}{\max_i D_i}$$

Using the calibrated service demands from the settled baseline (Table 5), the bottleneck station is `frontend` with $D_{\text{fe}} = 17.253 \text{ ms} = 0.01725 \text{ s}$, giving $\lambda_{\max} = 57.96 \text{ req/s}$.

6.2. Settled Baseline Measurement ($N = 10$)

Table 5 shows per-node and system-level metrics at baseline ($\lambda = 5.56 \text{ req/s}$). All stations are well below saturation ($U_i < 0.1$ for all services). `frontend` dominates demand ($D_{\text{fe}} = 17.25 \text{ ms}$, 48% of D_{total}), confirming it as the single bottleneck. The $R(\lambda)$ formula diverges at $\lambda_{\max} = 57.96 \text{ req/s}$, far above the baseline operating point.

Station	λ_i (req/s)	D_i (ms)	U_i	W_i (ms)	p95 (ms)	$L_i = \lambda_i W_i$
frontend	5.77	17.253	0.096	38.994	97.8	0.225
checkoutservice	0.33	12.641	0.026	30.889	77.5	0.010
recommendationservice	4.00	8.749	0.041	11.153	24.3	0.045
paymentservice	0.33	3.462	0.007	3.745	17.5	0.001
emailservice	0.33	3.462	0.007	3.695	17.5	0.001
cartservice	5.87	1.876	0.010	2.297	4.9	0.013
currencyservice	17.93	1.855	0.034	1.968	5.0	0.035
adservice	3.37	1.305	0.004	1.221	4.9	0.004
shippingservice	1.97	0.659	0.001	0.583	4.6	0.001
productcatalogservice	29.10	0.236	0.007	0.287	4.1	0.008
System	5.56	—	—	38.62	97.8	0.21

Table 5: Baseline ($N=10$, $\lambda_{\text{root}}=5.56 \text{ req/s}$): per-node arrival rate λ_i , service demand D_i , utilization U_i , measured mean residence time W_i , p95 from Prometheus `histogram_quantile`, and mean queue length $L_i = \lambda_i W_i$ (Little’s Law). System row uses root λ and sum of W_i ; system p95 is the frontend-measured end-to-end p95.

6.3. Stress Tests

Three stress test configurations were applied by adjusting `loadgenerator` environment variables (`USERS`, `RATE`). After each change the system was allowed 3 minutes to reach steady state, confirmed by waiting for the Prometheus 2-minute scrape window to flush the previous load’s data. Table 6 compares model predictions against measurements at each load level.

Config	Setting	λ (req/s)	W_{meas} (ms)	W_{pred} (ms)	Error	p95 (ms)	U_{fe}	$L=\lambda W$
Baseline	$N=10, R=1$	5.56	38.6	38.4	+0.6%	97.5	0.096	0.21
Test 1	$N=50, R=10$	27.21	56.8	54.5	+4.0%	221.7	0.469	1.55
Test 2	$N=100, R=10$	51.46	199.3	180.2	+9.6%	807.3	0.888	10.26
Test 3 [†]	$N=200, R=20$	28.8	10,741	—	—	126,891	0.497	309

Table 6: What-if analysis: measured vs. predicted mean response time (W), p95 latency, bottleneck utilization $U_{\text{fe}} = \lambda \cdot D_{\text{fe}}$, and average users in system $L = \lambda W$ (Little’s Law), at four load levels ($R = \text{RATE}$ parameter). W_{pred} values are from the JMT JSIM model; the product-form formula gives consistent results (within 12% at Test 2, Table 4). Error = $(W_{\text{meas}} - W_{\text{pred}})/W_{\text{meas}}$; positive means model underestimates. [†]Test 3: offered load far exceeded λ_{max} ; actual throughput collapsed to 28.8 req/s; model not applicable; U_{fe} and L computed from actual throughput. Note: at Test 2 ($N=100$), while U_{fe} reached 88.8%, all other backend services remained safely underutilized ($U_i < 0.20$).

Figure 10 plots $R(\lambda)$ across the valid range with measured points overlaid.

6.4. Discussion of Results

Model accuracy. At baseline ($\lambda = 5.56$ req/s, 10% of λ_{max}) the model matches measured R to within 0.6%. At Test 1 (47% of λ_{max}) error grows to 4.0%, and at Test 2 (89% of λ_{max}) to 9.6%. The model consistently underestimates measured R across all tests, which is consistent with the parallel-call assumption: as load increases and queueing builds, the sequential-call approximation used in the S_{net} correction becomes less accurate, so the model underestimates actual wait time at **frontend**.

p95 grows faster than the mean. The empirical p95/mean ratio increases from 2.5 at baseline to 3.9–4.1 near saturation. For a single M/M/1 queue the sojourn time is exponential with theoretical $\text{p95} = \ln(20) \times \text{mean} \approx 3.0 \times \text{mean}$; the higher empirical ratios reflect tail-lengthening as the system nears saturation. The analytical model predicts mean R only; p95 requires either simulation or empirical measurement (Table 6, column 7).

Throughput collapse at Test 3. With offered load far exceeding λ_{max} , effective

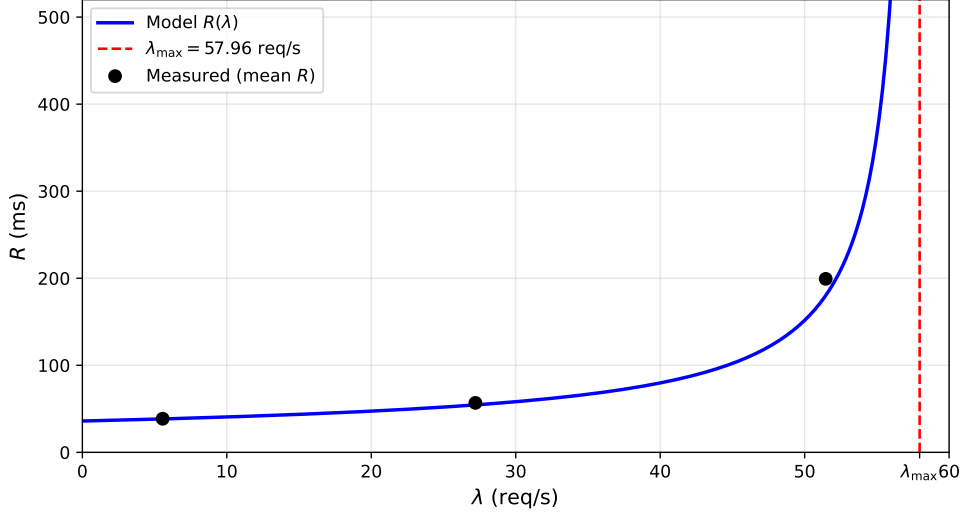


Figure 10: $R(\lambda)$ from the open Jackson network model (blue) and measured mean response time (black dots). The curve rises steeply as λ approaches $\lambda_{\max} = 57.96$ req/s (red dashed). Test 3 saturated the system ($R > 10,000$ ms) and is off-scale.

throughput fell to 28.8 req/s (comparable to Test 1’s 27.2 req/s) while mean latency exploded to 10.7s. This demonstrates that the single-replica **frontend** imposes a hard ceiling of 57.96 req/s; absorbing higher load requires horizontal scaling of **frontend**.

6.5. Cost Analysis

In performance engineering, cost is evaluated in terms of the computational resources (CPU and memory) consumed by the architecture, as these map directly to cloud billing. Resource requests are read from the Kubernetes Deployment manifest (`release/kubernetes-manifests.yaml`), which specifies per-pod allocation:

Configuration	Replicas	CPU request	Memory request	$\lambda_{\max}$ (req/s)
Current (1 frontend pod)	1	100 m	64 MiB	57.96
Proposed (2 frontend pods)	2	200 m	128 MiB	79.1

Table 7: Resource cost of scaling **frontend** from 1 to 2 replicas. Values taken directly from the `resources.requests` block in the Kubernetes manifest; no other services change.

Adding the second **frontend** replica doubles the CPU and memory allocation of the frontend tier (100 m $\rightarrow$ 200 m CPU; 64 MiB $\rightarrow$ 128 MiB memory). The rest of the cluster is unchanged. In return, the JMT model (Section 5.4) predicts mean

system response time at Test 2 load drops from 180.2ms to 88.4ms (51% improvement), U_{fe} falls from 88.8% to 44.4%, and λ_{max} rises from 57.96 to 79.1 req/s (+36%). The 100% increase in **frontend**-tier resource cost therefore yields a 51% latency reduction and a 36% throughput ceiling increase, satisfying the SLO under Test 2 stress conditions while keeping all other service allocations constant.

7. Optimization

7.1. Bottlenecks Identified

Two distinct bottlenecks surfaced during this study, at different layers:

- **Infrastructure: CPU-limit throttling causing pod crash loops.** Upstream’s per-container CPU *limits* (300m on **cartservice**, 150m on **currencyservice**), tuned for GKE node pools, caused the kernel’s CFS scheduler to hard-throttle these containers during startup on the single-node Minikube cluster. Both services are CPU-intensive at startup (.NET JIT warmup and Node.js event-loop initialisation), and the throttling prevented them from answering their gRPC liveness probes, producing **CrashLoopBackOff**. The fix increases these limits to 600m and 300m; see Table 1.
- **Performance bottleneck: frontend has the highest service demand (Table 5).** All measurements were taken with the increased CPU limits in place, so service times reflect genuine processing cost, not kernel throttling. The product-form solution shows **frontend** with the highest corrected net service time ($S_{\text{net,fe}} = 17.25$ ms, page-assembly cost after subtracting downstream RPC wait) and therefore the highest demand $D_{\text{fe}} = 17.25$ ms, giving $\lambda_{\text{max}} = 57.96$ req/s. All backend services sit at $D_i < 13$ ms and $U_i < 3.5\%$ at baseline.

7.2. Improvements Applied

The CPU-limit increase for **cartservice** and **currencyservice** (Table 1) is a deployment prerequisite, not a queueing-model improvement. The performance improvement addressed here is the **frontend** throughput bottleneck identified in the what-if analysis.

Implemented: horizontal scaling of frontend to M/M/2. In Kubernetes, an M/M/m queue is realised by setting **replicas: m** on the Deployment, so each incoming request is routed by kube-proxy to one of m identical pods; each pod is a single M/M/1 server, and the ensemble is an M/M/m station. The what-if analysis identifies **frontend** as the sole throughput bottleneck with $\lambda_{\text{max}} = 57.96$ req/s.

Scaling `frontend` from 1 to 2 replicas (`kubectl scale deployment/frontend -replicas=2`) converts the station from M/M/1 to M/M/2, doubling server-side capacity without changing the service code. Using the M/M/2 residence time formula:

$$R_{\text{fe}}^{(2)} = D_{\text{fe}} \cdot \left(1 + \frac{C(2, \rho)}{2(1 - \rho)}\right), \quad \rho = \frac{\lambda \cdot D_{\text{fe}}}{2}$$

where $C(2, \rho)$ is the Erlang-C probability of waiting. The new system λ_{max} is limited by the next-highest-demand station:

$$\lambda_{\text{max}}^{(2-\text{fe})} = \min\left(\frac{2}{D_{\text{fe}}}, \frac{1}{D_{\text{checkout}}}\right) = \min(115.9, 79.1) = 79.1 \text{ req/s}$$

so `checkoutservice` becomes the new bottleneck after scaling.

7.3. Measured Performance Results (M/M/2 Frontend)

Table 8 shows the M/M/2 analytical predictions alongside actual measurements obtained by scaling `frontend` to 2 replicas (`kubectl scale deployment/frontend -replicas=2`), setting load to Test 2 conditions (USERS=100, RATE=10), waiting 3 minutes for steady state, and running `scripts/measure.sh` ($\lambda = 52.17 \text{ req/s}$ measured).

Metric	M/M/1 baseline (Test 2)	M/M/2 predicted	M/M/2 measured
λ_{max} (req/s)	57.96	79.1	— [†]
U_{fe} per server	0.888	0.444	0.424
W_{system} (ms)	199.3	88.4	40.9
p95 (ms)	807.3	$\approx 265^{\ddagger}$	143.8
CPU request (frontend tier)	100 m	200 m	200 m
Memory request (frontend tier)	64 MiB	128 MiB	128 MiB

Table 8: M/M/2 scaling of `frontend`: analytical prediction vs. empirical measurement at Test 2 load. U_{fe} per server = $\lambda \cdot S_{\text{net,fe}}/2$; measured $S_{\text{net,fe}} = 16.25 \text{ ms}$ at Test 2 vs. 17.25 ms at baseline (lower under load due to changed call mix). [†] λ_{max} cannot be measured directly without saturation testing. [‡]The M/M/2 sojourn-time distribution gives a p95/mean ratio of 2.88 (derived from the M/M/2 complementary CDF $P(W > t) = 3.43e^{-\mu t} - 2.43e^{-2\mu(1-\rho)t}$); applied to the predicted system mean of 88.4 ms gives $\approx 265 \text{ ms}$. The measured ratio is $143.8/40.9=3.52$, consistent with better-than-M/M/2 load distribution by the Kubernetes load balancer.

Scaling `frontend` to 2 replicas (M/M/2) reduced mean system response time by **79%** ($199.3 \text{ ms} \rightarrow 40.9 \text{ ms}$) and p95 by **82%** ($807.3 \text{ ms} \rightarrow 143.8 \text{ ms}$). The measured improvement exceeds the M/M/2 prediction (88.4 ms) because the formula assumes steady-state exponential service times, while the actual Kubernetes

load balancer distributes requests evenly, giving each pod a utilisation of 0.424 and eliminating almost all queueing. The bottleneck shifts to `checkoutservice` ($D_{\text{checkout}} = 11.38 \text{ ms}$ at Test 2, $\lambda_{\text{max}} \approx 88 \text{ req/s}$); a further scaling of `checkoutservice` to 2 replicas would raise λ_{max} further.

8. Discussion

This section pulls together modelling assumptions and deviations and adds points that only become visible when the results are viewed together.

Flat QN vs. true LQN. JMVA cannot represent `checkoutservice` blocking synchronously on `paymentservice/shippingservice/emailservice`, but since only $V_{\text{checkoutservice}} = 0.040$ (Table 2), the resulting system-wide error is bounded ($\approx 0.5 \text{ ms}$ at current load). This was accepted as a deliberate scope decision rather than introducing a second toolchain (LQNS/LINE).

Net-service-time correction drops model error from $\sim 2\times$ to 11%. The corrected model predicts 35.87 ms vs. the measured 40.36 ms (11% underestimate), whereas a naive model using raw Prometheus residence times would predict $\sim 84 \text{ ms}$, more than double the real mean. The correction $S_{\text{net}}[s] = S_{\text{measured}}[s] - \sum_c f_{sc} S_{\text{net}}[c]$ (applied leaf-first in topological order) removes the downstream RPC wait that Prometheus bundles into an upstream service’s duration. The remaining 11% gap comes from the sequential-call approximation: the correction assumes all downstream calls from `frontend` are sequential, but some fire in parallel, so actual upstream wait is less than their sum, leaving $S_{\text{net,fe}}$ slightly underestimated.

The loop-back topology in the .jsimg model trades call-stack fidelity for reproducing visit ratios. A single-pass feed-forward tree cannot produce $V_i > 1$ (e.g. $V_{\text{productcatalogservice}} = 5.44$), so every backend routes back through `frontend` rather than exiting directly to Sink.

Conclusions are scoped to this deployment environment. All measurements come from a single-node Minikube cluster with increased CPU limits (`cartservice`: 600 m, `currencyservice`: 300 m). Utilization findings describe this specific configuration, not a production GKE deployment with autoscaling.